

PROJET DE RECHERCHE — CMI INFORMATIQUE

Création d'un Assistant Intelligent pour les Cours de Première Année

Public cible : Tout le monde

Encadrant : Yannick Estève

A. Louvet L. Ouelhadj T. De Faveri-Serre C. Cloupet H. Baudey

Années universitaires 2025–2027

Ce projet vise d'une part à apporter des contributions de recherche sur les LLM, le RAG, la recherche vectorielle et à en expliquer les principes indépendamment du niveau de familiarité du lecteur avec ces concepts, et d'autre part à construire un assistant conversationnel fondé sur la Génération Augmentée par Récupération (RAG), capable de répondre aux questions des étudiants à partir de leurs propres documents de cours.

Table des matières

1. État de l’art : Définitions et contexte

1.1. Poser les bases de l’Intelligence Artificielle

1.1.1 Bref historique

L’idée de machines capables de penser n’est pas nouvelle. Dès 1950, Alan Turing proposa son célèbre *Test de Turing* comme critère d’intelligence artificielle [?] : si une machine peut soutenir une conversation indiscernable de celle d’un humain, on peut, dans un certain sens, dire qu’elle pense. Le terme « Intelligence Artificielle » fut lui-même forgé en 1956 par John McCarthy lors de la conférence de Dartmouth, considérée comme l’acte fondateur du domaine.

Les décennies suivantes alternèrent entre périodes d’optimisme et « hivers de l’IA » — des intervalles de réduction des financements et de l’intérêt, causés par l’écart entre les promesses ambitieuses et les résultats modestes. Le paysage se transforma radicalement dans les années 1980 et 1990 avec l’essor de l’*apprentissage automatique* : plutôt que de coder manuellement des règles dans un programme, les chercheurs commencèrent à concevoir des systèmes capables d’*apprendre* des patterns directement à partir des données. Ce changement de paradigme posa les fondations de tout ce qui suivit.

Le véritable tournant survint en 2012, lorsqu’un réseau de neurones profond nommé AlexNet [?] écrasa la concurrence lors du challenge de reconnaissance visuelle ImageNet, démontrant que l’*apprentissage profond* — des réseaux de neurones à nombreuses couches entraînés sur de larges jeux de données — pouvait atteindre des performances bien supérieures aux méthodes classiques. À partir de ce moment, l’apprentissage profond devint l’approche dominante dans presque tous les sous-domaines de l’IA.

1.1.2 Qu’est-ce que l’Intelligence Artificielle ?

Le terme *Intelligence Artificielle* est l’une des expressions les plus utilisées — et les plus galvaudées — du paysage technologique actuel. Avant d’aborder les systèmes au cœur de ce projet, il convient de faire un pas en arrière pour comprendre ce qu’est réellement l’IA, d’où elle vient et pourquoi elle est devenue si centrale dans l’informatique moderne.

Dans sa définition la plus large, l’Intelligence Artificielle désigne tout système informatique conçu pour accomplir des tâches qui nécessiteraient normalement l’intelligence humaine — reconnaître des objets dans une image, comprendre une phrase parlée ou décider du prochain coup dans une partie d’échecs. Une définition plus précise, proposée par Russell et Norvig dans leur manuel de référence [?], décrit un agent IA comme un système qui *perçoit son environnement par des capteurs et agit sur lui par des actionneurs afin de maximiser une mesure de performance*.

Une distinction utile est celle entre l’*IA étroite* (ou IA faible) et l’*IA générale* (ou IA forte). L’IA étroite — qui couvre quasiment tous les systèmes existants — est conçue pour exceller dans un type de tâche spécifique : un filtre anti-spam classe des e-mails, un moteur de recommandation suggère des films, un modèle de langage génère du texte. L’IA générale, en revanche, serait un système capable de raisonner sur des domaines arbitraires au niveau humain ; elle reste en grande partie hypothétique et constitue un sujet de recherche à long terme. Tous les systèmes IA évoqués dans ce projet relèvent sans exception de la catégorie de l’IA étroite.

1.2. Les Grands Modèles de Langage (LLM)

Les grands modèles de langage, ou LLM (de l’anglais *Large Language Models*), sont bien plus présents dans notre quotidien qu’on ne le pense, ou même qu’on ne l’anticipait il y a quelques années encore. Mais que sont-ils vraiment ? Que recouvrent-ils, et à quoi servent-ils ? [?]

1.2.1 Qu'est-ce qu'un Grand Modèle de Langage ?

Un grand modèle de langage est, pour le dire simplement, un modèle mathématique qui génère du texte — comme remplir le mot manquant dans une phrase, un peu à la manière des « suggestions de mots » qui apparaissent en haut de votre clavier quand vous tapez un message sur votre téléphone. Mais ne vous y trompez pas : si votre téléphone peine parfois à prédire autre chose que votre commande de café habituelle, un LLM opère à une échelle franchement vertigineuse. Imaginez prendre chaque livre, article et fragment de code disponible sur internet, et les injecter dans un cerveau numérique qui cartographie les relations complexes entre chaque mot et chaque concept. [?]

En calculant la probabilité du mot suivant à partir de milliards de paramètres, ces modèles dépassent le simple « remplissage automatique » pour s'aventurer dans le domaine du raisonnement et de la créativité. Ils ne devinent pas des lettres — ils saisissent les nuances de l'intention humaine. Au cœur de ce processus se trouve une architecture appelée *Transformer* [?]. Pensez-y comme un système qui permet au modèle d'analyser un paragraphe entier d'un seul coup, plutôt qu'un mot à la fois. Ce mécanisme d'*attention* lui permet de se souvenir du début de votre histoire pendant qu'il en écrit la fin, maintenant un fil conducteur cohérent qui semble remarquablement humain.

1.2.2 À quoi servent les LLM ?

Parce qu'ils excellent dans la reconnaissance de patterns, leur utilité s'étend bien au-delà de répondre à des questions triviales ou de rédiger des e-mails :

- **Partenaires créatifs** : ils peuvent générer des slogans marketing ou rédiger de la poésie dans le style d'un romancier du XIX^e siècle.
- **Assistants techniques** : ils traduisent des langages de programmation complexes aussi facilement qu'ils traduisent l'anglais en français.
- **Assistants pédagogiques** : ils peuvent expliquer des concepts complexes, répondre à des questions et adapter leur registre au niveau du lecteur — ce qui est précisément le cas d'usage exploré dans ce projet.

Cette polyvalence comporte cependant des limites importantes, que nous explorons dans la section suivante.

1.2.3 Limites : hallucinations et frontières de la connaissance

Malgré leurs performances impressionnantes, les LLM souffrent de plusieurs limites bien documentées. La plus critique dans un contexte éducatif est leur tendance à produire des *hallucinations* — générer un texte fluide et plausible mais factuellement incorrect ou sans fondement [?, ?, ?].

Les hallucinations résultent de plusieurs causes. Premièrement, les LLM s'appuient sur des corrélations statistiques dans leurs données d'entraînement plutôt que sur un raisonnement factuel ancré. Deuxièmement, leur connaissance est statique : ils ont une date de coupure d'entraînement et ne peuvent accéder à des informations nouvelles ou spécifiques à un domaine que si celles-ci leur sont explicitement fournies. Troisièmement, lorsqu'une requête dépasse leur zone de connaissance certaine, ils peuvent fabriquer une réponse plutôt qu'admettre leur ignorance. Selon une étude de 2024 publiée dans l'*ACM Transactions on Information Systems*, les hallucinations se répartissent en deux grandes catégories : les *hallucinations de factualité* (affirmations incorrectes) et les *hallucinations de fidélité* (contenu incohérent avec une source donnée).

Ces limites sont particulièrement problématiques dans un contexte éducatif, où la précision est essentielle. Un assistant qui fournit avec assurance une information erronée sur une notion de cours peut activement induire les étudiants en erreur. C'est précisément ce constat qui motive le recours à l'ancrage dans des documents externes, via le RAG.

1.2.4 Biais et dégradation par rétroaction : les limites du RLHF

Une limite plus subtile mais significative concerne le mécanisme de retour utilisé pour améliorer les LLM [?]. Des techniques telles que le *Reinforcement Learning from Human Feedback* (RLHF) et l'IA Constitutionnelle sont employées pour aligner le comportement des modèles sur les préférences humaines et réduire les sorties nuisibles. Ce système repose sur la notation des réponses du modèle par les utilisateurs, le modèle étant mis à jour en conséquence [?]. Bien que ce processus vise à aligner le comportement du modèle sur les attentes des utilisateurs, il peut produire des effets pervers lorsque ceux-ci soumettent régulièrement des requêtes mal formulées.

Lorsqu'un utilisateur fournit une requête ambiguë, incomplète ou incohérente, le modèle n'a pas de base fiable pour produire une réponse pertinente. Il génère alors une réponse statistiquement plausible au regard de l'entrée, mais qui ne peut correspondre à l'intention non exprimée de l'utilisateur. Celui-ci, percevant la réponse comme hors sujet ou absurde, lui attribue une note négative — alors que, du point de vue du modèle, la réponse constituait un complément raisonnable à la requête donnée. Ce décalage entre le signal (une note négative) et sa cause réelle (une requête mal formulée plutôt qu'une erreur du modèle) introduit une *supervision corrompue* dans la boucle d'entraînement.

Répété à grande échelle, ce phénomène conduit le modèle à associer certains patterns de réponse légitimes à une récompense négative et à les désapprendre, entraînant une dégradation mesurable de la qualité générale des réponses. Une étude de Chen et al. [?] a documenté empiriquement ce phénomène pour ChatGPT, observant des changements comportementaux significatifs entre les versions du modèle, certaines tâches étant mieux réussies par les versions antérieures. Les auteurs attribuent en partie cette régression aux effets cumulatifs du bruit dans les retours utilisateurs au sein du pipeline RLHF.

Ce problème est particulièrement pertinent dans un cadre éducatif, où les étudiants — surtout en première année — peuvent manquer de la maîtrise de la formulation de requêtes nécessaire pour interagir efficacement avec le modèle. Un assistant qui se dégrade sous l'effet de retours bruités pourrait ainsi devenir progressivement moins utile pour son public cible, soulignant l'importance de concevoir des mécanismes robustes de collecte de retours et, autant que possible, de réduire la dépendance aux évaluations brutes des utilisateurs pour les mises à jour du modèle.

1.3. Récapitulatif

Un point de départ naturel pour cette vue d'ensemble est une question qui semble simple mais s'avère en réalité assez nuancée : qu'est-ce exactement qu'un LLM, et en quoi diffère-t-il de la notion plus large d'« IA » ? Le terme *Intelligence Artificielle* (IA) est un concept générique englobant toute technique ou tout système permettant aux machines d'imiter l'intelligence humaine, incluant l'apprentissage automatique, la vision par ordinateur, la robotique et le traitement du langage naturel. Un *Grand Modèle de Langage* (LLM), en revanche, désigne une catégorie spécifique de modèle IA conçu pour comprendre et générer du langage humain. Les LLM sont un sous-ensemble de l'IA, tout comme l'*Apprentissage Automatique* (ML) ou l'*Apprentissage Profond* (DL), etc. : si tous les LLM sont des systèmes IA, la réciproque est fausse.

Concrètement, les LLM sont des réseaux de neurones entraînés sur de vastes corpus textuels par apprentissage auto-supervisé, fondés sur l'architecture Transformer introduite par Vaswani et al. [?], qui permet aux modèles de capturer des dépendances à longue portée dans le texte grâce à des mécanismes d'auto-attention. L'introduction successive de la série GPT d'OpenAI — GPT-2 en 2019, GPT-3 en 2020, puis GPT-4 — a démontré que l'augmentation de la taille des modèles et des données d'entraînement engendre des *capacités émergentes* en compréhension et génération du langage [?] : des aptitudes qui n'apparaissent qualitativement qu'au-delà d'un certain seuil d'échelle. Les LLM contemporains tels que GPT-4, Claude d'Anthropic, LLaMA de Meta, Gemini de Google et Mistral font preuve d'une remarquable compétence dans un large éventail de tâches — de la réponse à des questions et du résumé à la génération de code et

au raisonnement en plusieurs étapes — et sont caractérisés par des nombres de paramètres de l'ordre de dizaines ou de centaines de milliards.

1.4. La Génération Augmentée par Récupération (RAG)

1.4.1 Principe et motivation

Avant de plonger dans l'architecture technique, il convient de faire un pas en arrière pour expliquer ce qu'est le RAG et pourquoi il a été développé, afin de rendre cette recherche accessible et intéressante pour tous.

Imaginez demander à un ami très cultivé de répondre à une question sur un sujet précis — disons, le contenu d'un cours auquel il n'a jamais assisté. Aussi intelligent soit-il, il ne peut tout simplement pas savoir ce qui a été enseigné dans ce cours particulier. Un LLM fait face à exactement la même limitation : il ne connaît que ce qu'il a vu pendant son entraînement et n'a aucun moyen de consulter des documents qu'il n'a jamais rencontrés. Imaginez maintenant qu'avant de répondre, votre ami soit autorisé à feuilleter rapidement les notes de cours pertinentes et à s'appuyer sur ce qu'il y trouve pour formuler sa réponse. C'est, en essence, ce que fait le RAG : il donne au modèle accès à un ensemble spécifique de documents au moment de répondre, de sorte que sa réponse soit ancrée dans ces sources plutôt que dans sa mémoire générale (potentiellement dépassée ou incomplète).

Plus formellement, la Génération Augmentée par Récupération (RAG) est un paradigme qui améliore les LLM en les connectant à une base de connaissances externe au moment de l'inférence [?]. Plutôt que de s'appuyer uniquement sur la mémoire paramétrique du modèle, un système RAG récupère d'abord les passages de documents pertinents depuis un corpus curé en fonction de la requête de l'utilisateur, puis passe ces passages en contexte au LLM, qui génère une réponse ancrée dans ces informations.

Cette approche répond directement aux limites décrites dans la section précédente : elle fournit au LLM des informations à jour et spécifiques à un domaine, et ancre la génération dans des sources explicites et vérifiables — réduisant significativement le risque d'hallucination. Une étude spécifiquement centrée sur le RAG pour les applications éducatives [?] confirme que cette approche améliore substantiellement la pertinence et la précision factuelle des réponses générées par l'IA dans des contextes académiques.

1.4.2 Architecture du RAG

Un pipeline RAG standard se décompose en trois étapes :

1. **Indexation (phase développeur)** : les documents sources (PDF, diapositives de cours, notes de cours, etc.) sont découpés en morceaux (*chunks*), encodés sous forme de vecteurs d'*embeddings* par un modèle d'embedding, et stockés dans une base de données vectorielle. (cf. Section 1.5 pour les détails sur la recherche vectorielle et l'indexation).
2. **Récupération (phase utilisateur)** : lorsqu'un utilisateur soumet une requête, celle-ci est encodée dans le même espace vectoriel. Le système effectue alors une recherche par similarité pour récupérer les k morceaux les plus pertinents depuis la base de données.
3. **Génération** : les morceaux récupérés sont ajoutés à la requête de l'utilisateur en tant que contexte dans le prompt du LLM. Le LLM génère ensuite une réponse ancrée dans ces informations récupérées.

1.4.3 Variantes et évolution du RAG

Le domaine a considérablement mûri depuis l'article fondateur sur le RAG. Des études de 2024–2025 identifient trois paradigmes principaux [?] :

- **RAG naïf** : le pipeline basique récupérer-puis-générer décrit ci-dessus.
- **RAG avancé** : introduit la réécriture des requêtes, le ré-ordonnancement des résultats récupérés et la récupération itérative pour améliorer la précision.
- **RAG modulaire** : une architecture plus flexible où chaque composant (récupérateur, lecteur, mémoire, etc.) peut être remplacé ou optimisé indépendamment.

Des travaux plus récents explorent également le **RAG agentique**, où le modèle peut décider *quand* récupérer et *quelles* sources consulter en fonction de la complexité de la requête, en utilisant une boucle de raisonnement multi-étapes [?].

Pour notre projet, une architecture RAG naïve ou avancée est suffisante compte tenu du périmètre bien défini et délimité du corpus documentaire (ressources de cours de première année).

1.5. Recherche vectorielle et indexation

1.5.1 Qu'est-ce qu'un embedding ?

Un *embedding*, dans le contexte de l'IA et des LLM, désigne la transformation d'un élément d'information — un mot, une phrase, un document, voire parfois une image — en une séquence de nombres réels appelée *vecteur*. Cette représentation numérique permet à la machine de manipuler le langage et les concepts non plus comme du texte symbolique, mais comme des objets mathématiques sur lesquels elle peut effectuer des calculs.

Plutôt que de « comprendre » les mots comme le fait un humain, le modèle les place dans un *espace vectoriel* de haute dimension (typiquement entre 384 et 4096 dimensions selon le modèle), où chaque élément occupe une position précise. Dans cet espace, la *distance* entre deux vecteurs reflète leur *similarité sémantique* : deux phrases au sens proche auront des vecteurs voisins, même si elles ne partagent aucun mot en commun. C'est ce principe qui permet à un système de récupérer, par exemple, le passage « *la photosynthèse permet aux plantes de produire de l'énergie* » à partir de la requête « *comment les plantes génèrent-elles leur énergie ?* ».

1.5.2 Comment les embeddings sont-ils appris ?

Ces vecteurs ne sont pas choisis arbitrairement : ils sont *appris* pendant l'entraînement du modèle. Le système lit des quantités énormes de texte et ajuste progressivement ses paramètres de sorte que les vecteurs résultants capturent les régularités du langage. Par exemple, le modèle

apprend que « chat » et « chien » apparaissent dans des contextes similaires et finit par leur attribuer des vecteurs voisins, tandis que « voiture » se retrouvera plus éloigné.

Des travaux pionniers comme *Word2Vec* [?] et *GloVe* [?] ont montré que ce principe simple suffit à faire émerger des relations sémantiques remarquables. L'exemple canonique d'arithmétique vectorielle, $\vec{\text{roi}} - \vec{\text{homme}} + \vec{\text{femme}} \approx \vec{\text{reine}}$, illustre que l'espace appris encode des concepts (genre, royauté) plutôt que de simples co-occurrences.

Dans les modèles modernes fondés sur les Transformers, les embeddings sont également *contextuels* : le vecteur associé à un mot dépend de la phrase dans laquelle il apparaît. Le mot « banque » dans « *je me suis assis au bord de la banque du fleuve* » et dans « *j'ai déposé de l'argent à la banque* » reçoit ainsi deux représentations distinctes, ce qui résout naturellement les ambiguïtés lexicales. Pour la recherche sémantique, on s'appuie généralement sur des modèles spécialisés comme ceux de la famille **sentence-transformers** [?], qui produisent un *vecteur unique* par phrase ou paragraphe, optimisé pour la comparaison par similarité plutôt que pour la génération de texte.

1.5.3 Mesurer la similarité entre vecteurs

Une fois le contenu représenté sous forme de vecteurs, comparer deux morceaux de texte revient à comparer deux points dans un espace. Trois métriques sont couramment utilisées :

- **La similarité cosinus**, qui mesure l'angle entre deux vecteurs et est insensible à leur magnitude : de loin la métrique la plus utilisée en recherche sémantique.
- **Le produit scalaire**, plus rapide à calculer mais sensible aux normes des vecteurs ; préféré lorsque les vecteurs ont été normalisés au préalable.
- **La distance euclidienne**, intuitive mais souvent moins discriminante en très haute dimension en raison du phénomène dit du *fléau de la dimensionnalité*.

1.5.4 Indexation et recherche approchée des plus proches voisins

Lorsqu'un corpus contient des milliers voire des millions de passages indexés, comparer la requête contre chaque vecteur stocké — la recherche dite *exacte* ou « force brute » — devient prohibitivement coûteuse. La solution standard consiste à s'appuyer sur des structures d'*indexation* permettant la *recherche approchée des plus proches voisins* (ANN, *Approximate Nearest Neighbor*) : on accepte une légère perte de précision en échange d'un gain de vitesse de plusieurs ordres de grandeur.

Plusieurs algorithmes d'indexation sont devenus des références :

- **IVF (Index à Fichier Inversé)** : partitionne l'espace en cellules via une étape préalable de clustering (typiquement *k-means*), puis restreint la recherche aux cellules les plus proches de la requête.
- **HNSW (Hierarchical Navigable Small World)** [?] : construit un graphe multicouches dans lequel la navigation locale converge rapidement vers le voisinage du vecteur recherché. C'est actuellement l'algorithme de référence dans la plupart des bases de données vectorielles, grâce à son excellent compromis entre rappel et latence.
- **Quantification par produit (PQ)** : compresse les vecteurs en sous-quantifiant chaque sous-espace, réduisant drastiquement l'usage mémoire au prix d'une légère perte de précision.

Ces algorithmes sont implémentés dans des bibliothèques et bases de données dédiées — les *bases de données vectorielles* — dont les plus utilisées dans l'écosystème open-source sont **FAISS** (Facebook AI Similarity Search), **Chroma**, **Qdrant** et **Weaviate**. Le choix dépend principalement du volume de données, des contraintes de latence et du besoin de filtrage par métadonnées (par exemple, restreindre la recherche à un cours ou un chapitre spécifique).

1.5.5 Synthèse

En pratique, les embeddings sont le bloc de construction fondamental de tout système RAG : ils servent à interpréter les requêtes, à comparer les passages de cours, à effectuer la recherche sémantique et, plus généralement, à alimenter les mécanismes internes des LLM. On peut les voir comme une « traduction » du langage humain en une géométrie de concepts, où les idées sont organisées dans un espace multidimensionnel que la machine peut explorer et exploiter efficacement. C'est précisément cette géométrie, combinée à un index adapté, qui permettra à notre assistant pédagogique de récupérer en quelques millisecondes les passages de cours pertinents pour répondre à la question d'un étudiant.

2. Présentation du projet

2.1. Objectifs pédagogiques

L'objectif est de découvrir les principes fondamentaux de l'intelligence artificielle moderne à travers les grands modèles de langage (LLM) et le système RAG (Génération Augmentée par Récupération). Les étudiants apprendront à interfacer un LLM avec une base documentaire personnalisée afin de créer un assistant conversationnel capable de répondre à des questions portant sur leurs cours.

2.2. Les assistants IA dans les environnements éducatifs

2.2.1 Panorama des travaux existants

L'application d'assistants alimentés par l'IA à l'enseignement suscite un intérêt de recherche croissant. Une étude publiée dans *Applied Sciences* [?] a recensé 47 travaux portant sur des chatbots éducatifs fondés sur le RAG, dont la majorité publiés en 2024, témoignant de la croissance rapide de ce domaine.

Plusieurs déploiements réels sont directement pertinents pour notre projet :

- **Assistant UniMiB** (Université de Bicocca, Milan, 2024) [?] : un assistant étudiant fondé sur le RAG, capable de répondre à des questions sur les procédures universitaires, les emplois du temps et les informations administratives. Il indexe les documents officiels de l'université et utilise un LLM pour générer des réponses ancrées dans ces sources.
- **OwlMentor** (*Frontiers in Psychology*, 2024) [?] : un environnement d'apprentissage assisté par l'IA conçu pour aider les étudiants universitaires à comprendre des textes scientifiques. Intégré dans un cours, il proposait un chat fondé sur des documents, une génération automatique de questions et la création de quiz.
- **Assistant IA CS50** (Université Harvard) : intégré dans un cours d'introduction à l'informatique, un assistant fondé sur le RAG s'est révélé capable de fournir des réponses détaillées et contextuellement appropriées, facilitant un tutorat personnalisé à grande échelle.
- **Assistant RAG pour l'enseignement médical** (*npj Digital Medicine*, Nature, 2025) [?] : un assistant pédagogique déployé dans un cours de médecine, dont les patterns d'usage et les retours étudiants ont été analysés sur deux cohortes consécutives, démontrant la faisabilité et l'efficacité des assistants fondés sur le RAG dans des contextes éducatifs à enjeux élevés.

2.2.2 Efficacité et défis

Le constat général qui se dégage de ces études est que les assistants fondés sur le RAG sont efficaces pour fournir des réponses précises et ancrées dans des sources à des questions spécifiques à un domaine, et que les étudiants les perçoivent positivement comme un complément à l'enseignement humain. Plusieurs défis sont cependant régulièrement signalés :

- **Qualité et couverture des documents** : la qualité des réponses de l'assistant est directement bornée par la qualité et la couverture du corpus documentaire indexé. Des documents de cours mal formatés ou incomplets dégradent les performances.
- **Requêtes hors périmètre** : les étudiants posent souvent des questions en dehors du domaine des documents indexés. Les systèmes robustes doivent détecter ces requêtes et refuser poliment de spéculer.
- **Difficulté d'évaluation** : évaluer la correction des réponses générées dans des contextes éducatifs ouverts est non trivial et nécessite souvent une évaluation par des experts humains.

- **Calibration de la confiance des étudiants :** les étudiants peuvent sur-se-fier aux réponses de l'IA ou, au contraire, s'en méfier même lorsqu'elles sont correctes. Communiquer l'incertitude et citer les sources est important pour une calibration appropriée de la confiance.

2.3. Synthèse et positionnement de notre projet

Cet état de l’art révèle que la combinaison LLM + RAG représente l’état de l’art actuel pour construire des assistants conversationnels spécifiques à un domaine. La pile technologique que nous prévoyons d’utiliser — Python, un modèle d’embedding (`sentence-transformers` ou OpenAI), une base de données vectorielle (FAISS ou Chroma) et un framework d’orchestration (LangChain ou LlamaIndex) — est bien validée tant dans la littérature académique que dans des déploiements éducatifs réels.

La principale originalité de notre projet réside dans son périmètre spécifique : construire un assistant adapté aux *ressources de cours de première année du CMI Informatique*, qui présentent des caractéristiques particulières (informatique et mathématiques de niveau introductif, contenu de projet en français et en anglais). Les défis que nous anticipons s’alignent sur ceux rapportés dans la littérature : assurer une couverture documentaire suffisante, gérer les questions hors périmètre et calibrer la confiance des étudiants envers les sorties du système.

-> Lisa va présenter l'IA + parler des LLMs -> Camille va définir et parler des RAGs -> Thomas va parler de l'indexation / embeddings / vectorisation -> Arthur va parler des appels d'API / création du RAG avec rassemblement des ressources disponibles